Istruzioni: Scrivere immediatamente su ogni foglio cognome, nome e numero di matricola. È consentito utilizzare esclusivamente i fogli forniti dalla commissione. Tutto il materiale personale (quaderni, appunti, dispositivi elettronici), deve essere riposto nello zaino - che deve rimanere chiuso - o depositato sulla cattedra. I dispositivi elettronici (telefoni, smartwatch, fitness band, auricolari, tablet, ecc.) atti alla ricezione di informazioni devono essere spenti e non devono essere mai visibili durante la prova. La prova ha una durata di **3 ore**. Ogni violazione delle regole comporterà l'immediato annullamento della prova.

Esercizio 1

Vi è commissionata la realizzazione di StackGPT, un sistema basato su LLM progettato per supportare ingegneri del software in ogni fase del ciclo di sviluppo, dalla progettazione alla manutenzione del codice. StackGPT funziona come un assistente esperto in ingegneria del software, capace – tra le altre cose – di comprendere e generare codice in linguaggi di programmazione comuni. Gli utenti registrati, possono effettuare il login, accedere al sistema, e eventualmente modificare la password.

StackGPT permette agli utenti registrati di fare richieste inerenti all'Ingegneria del Software. Per fare una richiesta, l'utente indica un titolo e una descrizione dettagliata della richiesta, uno o più tag personalizzati, e seleziona tra due modelli di elaborazione: un modello avanzato (`StackGPT-4o`) ottimizzato per task complessi e un modello più leggero (`StackGPT-3.5`) ideale per task più semplici. Ogni utente ha a disposizione una quota giornaliera di 1M di token in input/output per il modello `StackGTP-3.5` e di 10k token in input/output per `StackGPT-4o`. Superata tale soglia, l'utente deve attendere che la quota si re-imposti, nel giorno successivo, per porre nuove richieste. Quando un utente inserisce una richiesta, la risposta di SlackGPT viene generata in tempo reale sfruttando le API di OpenAI (che ospita i due modelli utilizzati) e mostrata all'utente, e i token consumati vengono detratti dalla quota disponibile (che gli utenti possono visualizzare tramite una funzionalità apposita).

Le richieste degli utenti e le risposte generate dal sistema sono inoltre pubbliche (ogni utente registrato può visualizzare le richieste di ogni altro utente, effettuando filtraggi per parola chiave o per contenuto testuale). Gli utenti possono inoltre valutare le risposte del sistema (tramite "mi piace/non mi piace") e lasciare commenti per evidenziare eventuali incompletezze, inconsistenze o errori. Il sistema permette infine agli amministratori della piattaforma di modificare le quote di utilizzo token, e visualizzare i costi di utilizzo mensile in tempo reale, sfruttando le API di OpenAI.

Si svolgano i seguenti punti, usando la propria conoscenza del dominio per derivare dettagli non definiti nei requisiti.

a. Modellare tutti i requisiti funzionali del sistema descritto sopra utilizzando uno Use Case Diagram;
b. Realizzare i mock-up dell'applicazione descritta, relativamente alla funzionalità di inserimento di una richiesta.
c. Dettagliare il caso d'uso relativo alla funzionalità di inserimento di una richiesta, per mezzo descrizioni testuali strutturate secondo un formalismo tabellare come quello proposto da A. Cockburn.

Esercizio 2

L'*Art Institute of Chicago* vi ha appena assunto come consulenti esperti di Ingegneria del Software per supportarli nella gestione di una fornitura di totem interattivi multimediali (touchscreen). Questi dispositivi saranno distribuiti nelle sale espositive e nelle aree comuni del museo, con lo scopo di: fornire informazioni dinamiche su opere d'arte, mostre temporanee e servizi del museo; guidare i visitatori attraverso percorsi tematici; Supportare funzioni di accessibilità (es. descrizioni audio, traduzioni multilingue); integrarsi con il sistema gestionale del museo per aggiornamenti in tempo reale.

Vi è richiesto di contribuire alla redazione del documento di specifica dei requisiti software per la fornitura di cui sopra, fornendo almeno **cinque** requisiti **non funzionali**.

Esercizio 3

Un'agenzia di cybersecurity ha sviluppato un agente autonomo basato su AI che monitora forum e social media per identificare potenziali minacce (es. frodi, discorsi d'odio, traffico illecito, fake news).

Una volta attivato, l'agente inizia le attività di monitoraggio, scegliendo la prima pagina web da visitare in maniera causale da un pool predefinito di pagine iniziali. La pagina viene quindi caricata e analizzata, cercando contenuti critici. Se non vengono individuate criticità, l'agente sceglie, tra i link presenti nella pagina web corrente, la prossima pagina da visitare, e ripete il ciclo. Se invece vengono ravvisate criticità, il sistema raccoglie ulteriori informazioni di contesto (e.g.: date, luoghi) presenti nella pagina, e trasmette un report alla centrale operativa, prima di riprendere le regolari attività. In ogni momento, se si verificano problemi di connettività, l'agente mette in pausa le operazioni e, quando la connessione ritorna stabile, riprende le attività da dove erano state interrotte.

Si specifichi tramite uno Statechart il comportamento dell'agente AI descritto sopra.

Esercizio 4

Il metodo statico *calcolaPrezzo* della classe *ServiziMuseali* viene utilizzato per calcolare il costo di un biglietto di ingresso in una struttura museale. Il metodo accetta i seguenti parametri:

- *int age*: età della persona che vuole acquistare il biglietto;
- *String type*: stringa che indica il livello di fidelity del cliente. Può assumere valori in "BASE" (clienti regolari), "UNIVERSITY" (per studenti universitari), o "GUIDES" (per guide e accompagnatori);
- *boolean protected:* indica se il cliente appartiene ad una delle categorie protette a norma di legge;

Se almeno uno dei parametri è non valido, il metodo solleva una IllegalArgumentException. In caso contrario, il metodo ritorna un numero float rappresentante il prezzo (in Euro) del biglietto. Tale prezzo è calcolato in base alla tipologia di cliente e in base alla sua età, come da tabella seguente.

		age		
		$\boldsymbol{age < 18}$	$\boldsymbol{18 \leq age < 65}$	$\boldsymbol{age \geq 65}$
type	**BASE**	0	10	5
	UNIVERSITY	0	7	0
	GUIDES	0	5	0

Per le categorie protette, inoltre, il biglietto di ingresso è sempre gratuito.

a. Indicare, per ciascuno dei parametri del metodo *calcolaPrezzo*, un partizionamento in classi di equivalenza per testing black-box;
b. Scrivere tre test JUnit con strategia Black Box per il metodo *calcolaPrezzo*, indicando per ciascuno di essi quali classi di equivalenza copre. Si richiede inoltre che un test corrisponda a scenari in cui almeno uno dei parametri è non valido, e che i restanti due corrispondano a scenari in cui tutti i parametri sono validi. Ove possibile, i test devono coprire classi di equivalenza distinte.
c. Quanti test sono necessari per testare il metodo con strategia R-WECT? Motivare la risposta, descrivendo brevemente tale strategia.